

Consommation de l'API REST

1. Vue d'ensemble de l'Architecture

Le module de communication réseau de l'application Java est conçu pour interagir avec une API REST distante.

L'architecture repose sur une approche qui garanti le maintien de l'état , la persistance sécurisée des sessions, et une sérialisation dynamique des payloads.

Les opérations réseau utilisent l'API native `java.net.http.HttpClient` , combinée à **Jackson** (`ObjectMapper`) pour le mapping objet-relationnel (JSON/POJO).

2. Le Moteur HTTP : `ApiUtils`

La classe `ApiUtils` agit comme une classe statique centralisant toutes les requêtes (GET, POST, etc.) vers le backend.

2.1 Configuration du Client

L'instance de `HttpClient` est configurée au démarrage de l'application avec des paramètres stricts :

- **Timeouts** : Un timeout de connexion (ex: 10s) et un timeout de requête (ex: 30s) sont définis via `Duration.ofSeconds()` pour éviter le freeze de l'UI en cas de latence serveur.
- **Gestion des Cookies** : Le client intègre nativement un gestionnaire d'état via `.cookieHandler(new CookieManager(...))` . La politique d'acceptation est paramétrée (ex: `ACCEPT_ALL` ou restrictive au domaine) pour stocker automatiquement les jetons de session (Token/Session ID) renvoyés par l'API.

2.2 Routage et Traitement Générique (`execute`)

Le traitement des requêtes est factorisé via une méthode générique de type `<T> T execute(HttpRequest, Class<T>)` .

1. **Envoi** : Utilisation de `HttpResponse.BodyHandlers.ofString()` pour capturer la réponse brute en UTF-8.
2. **Désérialisation (Jackson)** : L'API répondant généralement via une enveloppe standard (ex: `ApiResponse<T>`), le type paramétré est reconstruit dynamiquement via `TypeFactory.constructParametricType()` .
3. **Gestion des Erreurs** : L'analyse du `statusCode` HTTP (200, 401, 500, etc.) déclenche la levée d'exceptions métier (ex: `ApiException`) si le code diffère de la plage de succès, en extrayant le message d'erreur du payload JSON.

3. Sécurité et Persistance de Session (`CookieUtils` & `CookieParsing`)

Étant une application client lourd, la session utilisateur doit persister entre les redémarrages. Ceci est géré de manière hautement sécurisée.

3.1 DTO et Mapping des Cookies

La classe `HttpCookie` native de Java n'étant pas facilement sérialisable, l'application utilise un DTO intermédiaire (`CookieParsing`). Ce DTO capture l'intégralité des attributs du cookie (`Name` , `Value` , `MaxAge` , `Domain` , `Path` , `HttpOnly` , `Secure`).

3.2 Persistance et Chiffrement

Pour prévenir le vol de token de session sur la machine physique de l'utilisateur :

1. **Sérialisation** : Le `CookieStore` est converti en tableau JSON via Jackson.
2. **Chiffrement** : Le payload JSON est envoyé à `EncryptUtils` qui chiffre la chaîne.
3. **Opérations** : L'artefact chiffré est écrit sur le disque dans un fichier `cookies.enc` .
4. **Shutdown Hook** : La méthode `save()` de `CookieUtils` est liée à un `Runtime.getRuntime().addShutdownHook(...)` garantissant l'écriture sécurisée des cookies même lors d'un arrêt abrupt de la JVM.

3.3 Restitution (Warm-up)

Au démarrage, `ApiUtils` invoque `CookieUtils.getInstance().loadCookies()` . Le fichier chiffré est lu, déchiffré, mappé en POJO, et réinjecté sous forme de `HttpCookie` dans l'espace mémoire du `CookieStore` . La communication avec l'API reprend ainsi l'état d'authentification exact de la session précédente.

4. Cache (`SessionUtils`)

Afin de minimiser les appels réseau redondants (réduire la charge serveur et la latence client), l'application utilise `SessionUtils` en tant que cache.

- **Modèle Stateful** : Après une authentification réussie ou un appel `/auth/me` , l'objet `User` (et d'autres contextes globaux comme `Challenge`) est conservé dans le Singleton `SessionUtils` .
- Les composants métiers et UI interrogent ce cache en priorité via `SessionUtils.getInstance().getUser()` plutôt que de refrapper l'API REST.

5. Cas Particulier : Upload de Fichiers

L'envoi de fichiers binaires (ex: images) via REST nécessite la construction manuelle de flux `multipart/form-data` (méthode `uploadEntry`).

- **Construction du Stream** : Un `ByteArrayOutputStream` est utilisé en mémoire.
- **Boundary** : Un séparateur unique (`---Boundary + timestamp`) est généré.

- **Data Aggregation** : L'entête de chaque partie est écrite via un `PrintWriter` . Les métadonnées (`userId` , `challengeId`) sont injectées en tant que texte.
- **Stream binaire** : Le binaire pur de l'image est transféré directement du `FileSystem` vers le flux HTTP via `Files.copy(file.toPath(), byteStream)` .
- **Publication** : Le body publisher `HttpRequest.BodyPublishers.ofByteArray()` expédie le payload complet.